```
template<typename T>
class lock_free_stack
{
private:
    struct node;

    struct counted_node_ptr
    {
        int external_count;
        node* ptr;
    };

    struct node
    {
        std::shared_ptr<T> data;
        std::atomic<int> internal_count;
        counted_node_ptr next;

        node(T const& data_):
            data(std::make_shared<T>(data_)),
             internal_count(0)
        {}
    };

    std::atomic<counted_node_ptr> head;

    void increase_head_count(counted_node_ptr& old_counter)
    {
        counted_node_ptr new_counter;

        do
        {
             new_counter=old_counter;
             ++new_counter.external_count;
         }

while(!head.compare_exchange_strong(old_counter,new_counter,

std::memory_order_acquire,

std::memory_order_relaxed));

          old_counter.external_count=new_counter.external_count;
    }

public:
    ~lock_free_stack()
     {
        while(pop());
     }
```